

How ROS ?

now that we have everything installed

The ROS 2 logo consists of a 3x3 grid of nine dark blue dots to the left of the text "ROS 2".

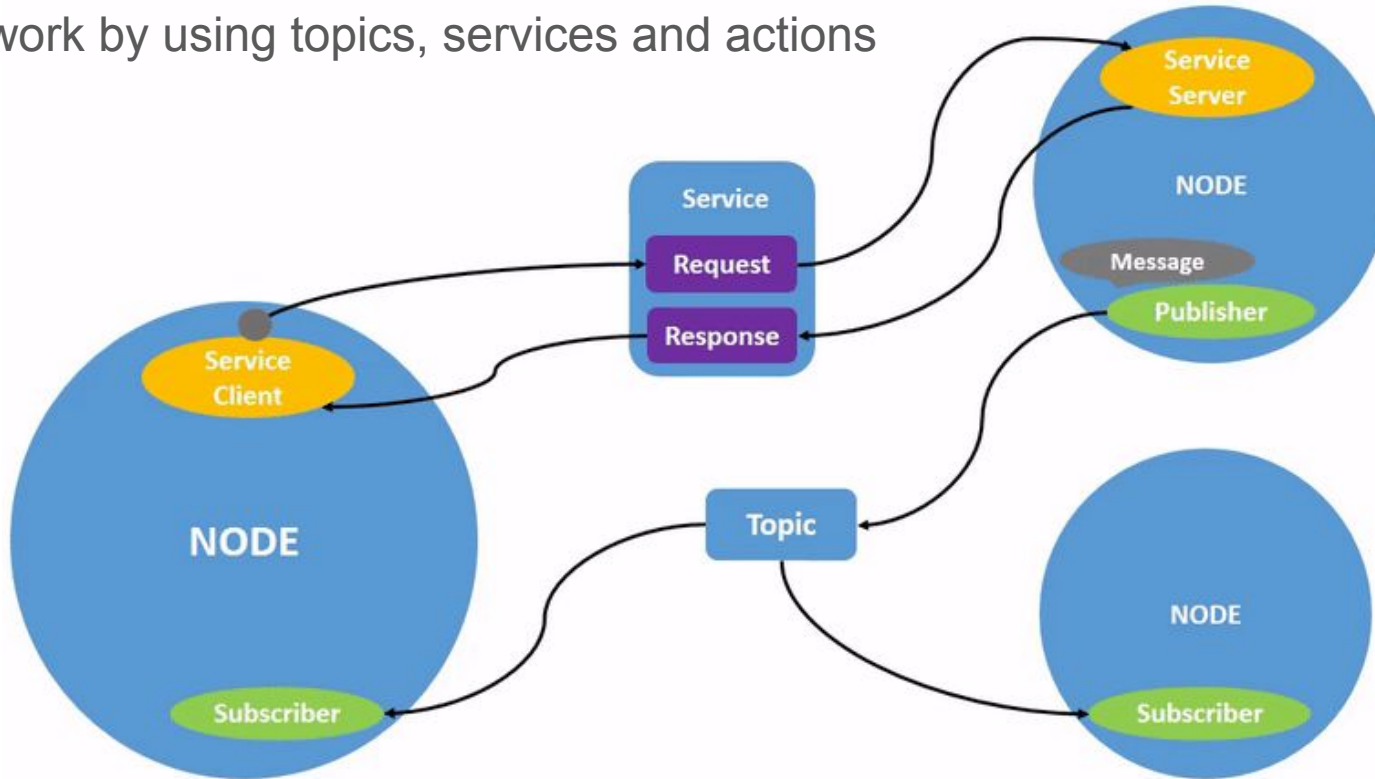
ROS 2

Node - what is a node ?

- A node executes code and communicates **with other nodes** in the ROS network by using topics, services and actions
- It is the smallest unit of processing in the ROS 2 ecosystem
- Each node in ROS **should** be responsible for a single, modular purpose, e.g. controlling the wheel motors or publishing the sensor data from a laser range-finder
- Nodes can communicate with other nodes within the **same** process, in a **different** process, or on a **different machine**
- Nodes use ROS client library to connect with other nodes

Node - what is a node ?

- A node executes code and communicates **with other nodes** in the ROS network by using topics, services and actions

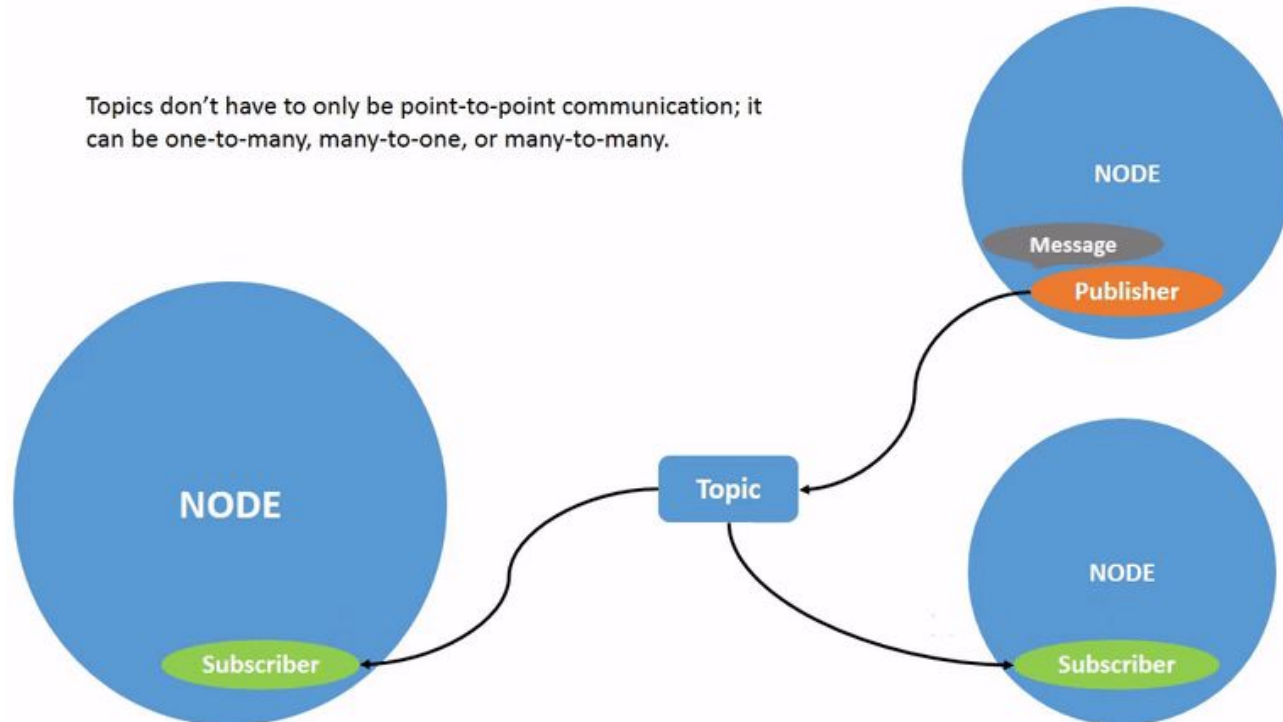


Topic - what is a topic ?

- A labeled channel for communication **between nodes** and are an implementation of a **Publish/Subscribe** communication pattern
- A node can provide information by **publishing a message** to a topic
- A node can receive information by **subscribing** to a topic
- One of the three primary styles of interfaces provided by ROS 2
- Should be used for continuous data streams, like sensor data, robot state, etc
- The publish/subscribe system is “**strongly-typed**”. A topic is declared with a message type and all data exchanged through that topic must conform to the structure defined by that message type

Topic - what is a topic ?

- A labeled channel for communication **between nodes** and are an implementation of a **Publish/Subscribe** communication pattern



Message - what is a message ?

- Bits of data sent from one node to another over a **topic**
- A way for a ROS 2 node to send data on the network to other ROS nodes, with no response expected
- Written and defined in the ROS Message Description Language (.msg files) in the msg/ directory of a ROS package
- Custom messages can be created if the user needs a new struct of data
- Many common messages are available in:
https://github.com/ros2/common_interfaces

Nodes, topics and messages - practice

- In one terminal, source the setup file and then run a C++ talker (i.e., a publisher):

```
source /opt/ros/humble/setup.bash  
ros2 run demo_nodes_cpp talker
```

- In another terminal, source the setup file and then run a Python listener (i.e., a subscriber):

```
source /opt/ros/humble/setup.bash  
ros2 run demo_nodes_py listener
```

- Tired of sourcing the setup file ? Add the command to your bashrc file:

```
echo "source /opt/ros/humble/setup.bash" >> .bashrc
```

Nodes, topics and messages - practice

- Explore these commands in other terminals:

```
ros2 topic list
```

```
ros2 topic echo /chatter
```

```
ros2 topic info /chatter
```

```
ros2 node list
```

```
ros2 node info /listener
```

```
ros2 node info /talker
```

- Want more details ? Try this one:

```
ros2 doctor --report (add | less in the end to see all info, key 'q' to escape)
```

TIP: Do not forget to source in each terminal

Package - what is a package ?

- The basic organization unit for ROS components that can be installed and uninstalled as a single unit
- Contains all the code for one specific feature (teleoperation, sensing, mapping, etc)
- It includes code, parameters, configuration files, dependencies, build instructions, interface descriptions.
- Distributed either as source code (e.g., a git repository) or as a binary file (e.g., a debian (.deb) file, what is installed with 'sudo apt install [...]')
- Some packages rely on other packages to function, and many ROS packages depend on packages that are distributed with Ubuntu

Package - where do we find a package ?

- Packages can be found in
 - [ROS Index](#)
 - Github
 - Try a `sudo apt search ros-$ROS_DISTRO-*` in your terminal ;)
- Package documentation is on GitHub or on the package's own website
- New packages announcements are usually sent in
 - [OpenRobotics Discourse - ROS](#)
 - LinkedIn, like [Open Robotics](#)

Package - how do we install a package ?

- Using as an example the [SLAM Toolbox package](#) (you **don't** need to install it now)
 - APT (Advanced Package Tool):

```
sudo apt install ros-$ROS_DISTRO-slam-toolbox
```
 - Docker:

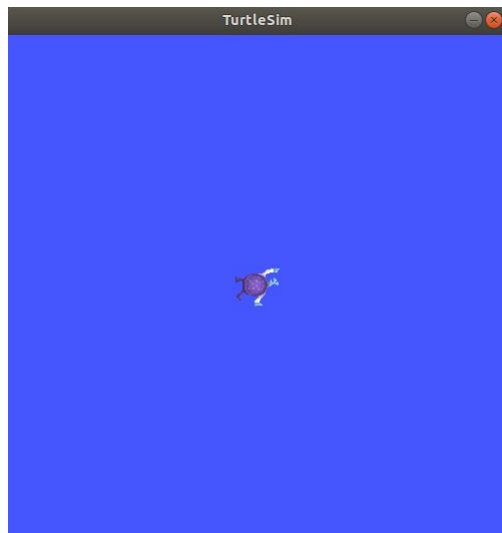
```
docker pull husarion/slam-toolbox:humble
```
 - Snap:

```
sudo snap install slam-toolbox --edge
```
 - Build from source with `colcon build`

Turtle time !

Let's practice with turtlesim:

<https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Introducing-Turtlesim/Introducing-Turtlesim.html>



Install Turtlesim

Install the turtlesim package for your ROS 2 distro:

```
sudo apt update
```

```
sudo apt install ros-$ROS_DISTRO-turtlesim
```

To check if the package is installed, run the following command, which should return a list of turtlesim's executables:

```
ros2 pkg executables turtlesim
```

Start it:

```
ros2 run turtlesim turtlesim_node
```

Using it:

TIP: Do not forget to source in each terminal

In a new terminal, verify the running nodes:

```
ros2 node list
```

Execute keyboard control:

```
ros2 run turtlesim turtle_teleop_key
```

Use the arrow keys on your keyboard to control the turtle.

RQT

TIP: Do not forget to source in each terminal

Install the RQT package for your ROS 2 distro:

```
sudo apt install ros-$ROS_DISTRO-rqt*
```

RQt makes it easier to manage all the various windows in a single screen layout.
Explore some tools:

```
rqt
```

```
ros2 run rqt_graph rqt_graph
```

```
ros2 run rqt_py_console rqt_py_console
```

Working with topics:

TIP: Do not forget to source in each terminal

In a new terminal, verify the running nodes:

```
ros2 topic echo /turtle1/cmd_vel
```

- Update the rqt_graph

See the message type that is being published in the topic:

```
ros2 topic type /turtle1/cmd_vel
```

Discover the message data:

```
ros2 interface show geometry_msgs/msg/Twist
```

Working with topics:

TIP: Do not forget to source in each terminal

Publishing msgs (`ros2 topic pub [topic] [msg_type] [args] )`

```
ros2 topic pub /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.2, y: 0.0, z: 0.0},  
angular: {x: 0.0, y: 0.0, z: 0.5}}"
```

Publishing msgs repeatedly:

```
ros2 topic pub -r 1 /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.3, y: 0.0, z:  
0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}"
```

Measuring the msg frequency:

```
ros2 topic hz /turtle1/pose
```

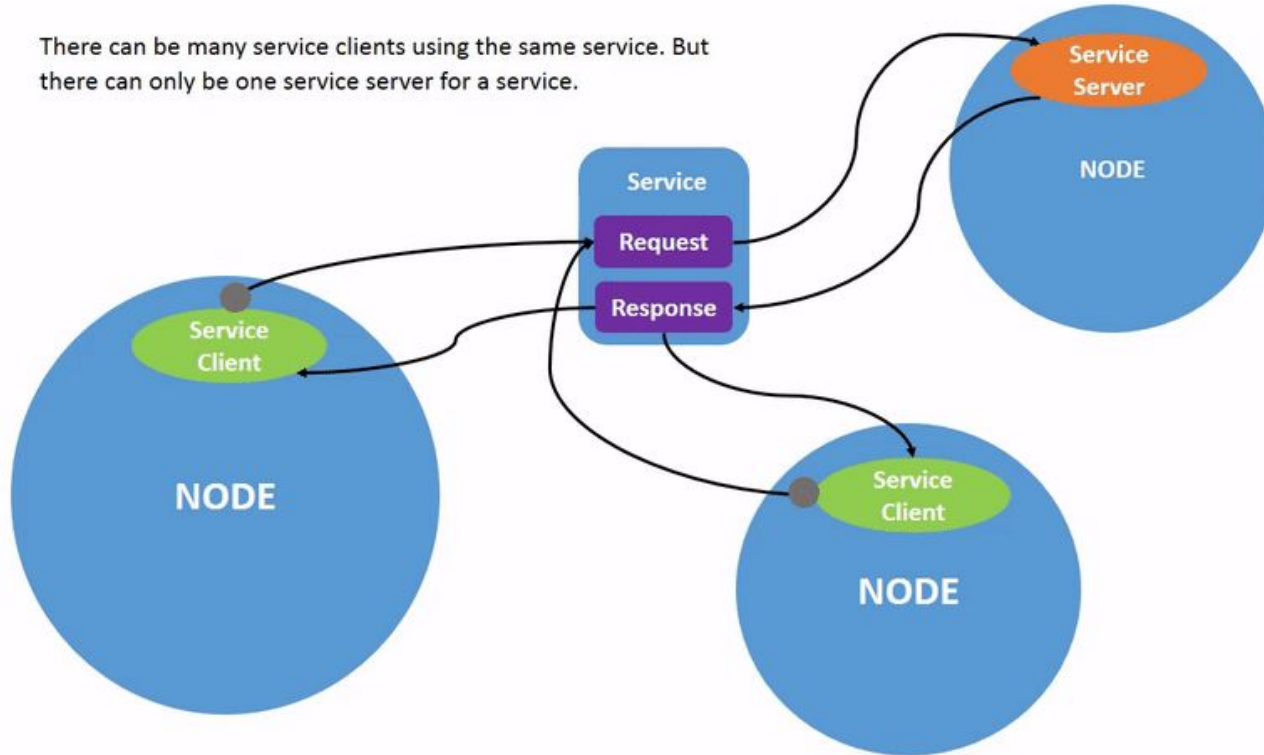
Service - what is a service ?

- Provide a **request/response** mechanism for **inter-node** communication
- Only provide data when they are **specifically** called by a client
- Expected to return **quickly**, as the client is generally waiting on the result
- Through services, a **node** can make a remote procedure call to another **node** which will do a computation and return a result
- Through services, the nodes are exposing their **instantaneous functionalities** (e.g., turn on a light, take a photo, get a parameter)
- A node provides a service through a **server**, and calls a service through a **client**

Service - what is a service ?

- Provide a **request/response** mechanism for **inter-node** communication

There can be many service clients using the same service. But there can only be one service server for a service.



Working with services:

TIP: Do not forget to source in each terminal

List all services:

```
ros2 service list
```

Service type:

```
ros2 service type /clear
```

Requiring a service | `ros2 service call [service] [args]`

```
ros2 service call /clear std_srvs/srv/Empty
```

Requiring a new turtle:

- Service type:
 - `ros2 interface show turtlesim/srv/Spawn`
- New turtle:
 - `ros2 service call /spawn turtlesim/srv/Spawn "{x: 8.0, y: 5.0, theta: 3.0, name: 'Eric'}"`

Remapping

TIP: Do not forget to source in each terminal

You need a second teleop node in order to control turtle Eric. However, if you try to run the same command as before, you will notice that this one also controls turtle1. The way to change this behavior is by remapping the cmd_vel topic.

```
ros2 run turtlesim turtle_teleop_key --ros-args --remap  
turtle1/cmd_vel:=Eric/cmd_vel
```

Parameter - what is a parameter ?

- A configuration value of a individual node
- Used to configure nodes at startup (and during runtime), without changing the code (i.e., no need to hard code)
- A node can store parameters as integers, floats, booleans, strings, and lists
- Can be accessed by all ROS nodes and modified by the user to change node behavior
- Each parameter consists of a key, a value, and a descriptor. The key is a string, the value is the data type (e.g., bool, float, int) and the descriptor contains value ranges, type information or additional constraints

Working with parameters:

TIP: Do not forget to source in each terminal

List all parameters:

```
ros2 param list
```

Consulting the simulator background green component:

```
ros2 param get /turtlesim background_g
```

Changing the background color:

```
ros2 param set /turtlesim background_r 150
```

Soliciting the simulator to clear:

```
ros2 service call /clear std_srvs/srv/Empty
```

Bag - what is a bag ?

- A bag is a format for storing message data
- Binary format with file extension *.bag
- Suited for logging and recording datasets for later visualization and analysis

Record all topics in a bag:

```
ros2 bag record --all
```

Record given topics:

```
ros2 bag record topic_1 topic_2 topic_3
```

Bag - what is a bag ?

Show information about a bag:

```
ros2 bag info bag_name.bag
```

Play given topics:

```
ros2 bag play bag_name.bag
```

Play chosen topics:

```
ros2 bag play bag_name.bag --topics topic1 topic2 topic3
```

Example: https://cvg.cit.tum.de/data/datasets/rqbd-dataset/file_formats

About bags: <https://docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Recording-And-Playing-Back-Data/Recording-And-Playing-Back-Data.html>

Bag - what is a bag ?

Playback options can be defined e.g

```
ros2 bag play --rate=0.5 bag_name.bag
```

- rate=factor Publish rate factor
- clock Publish the clock time (set param use_sim_time to true)
- loop Loop playback
- etc.

Example: https://cvg.cit.tum.de/data/datasets/rqbd-dataset/file_formats

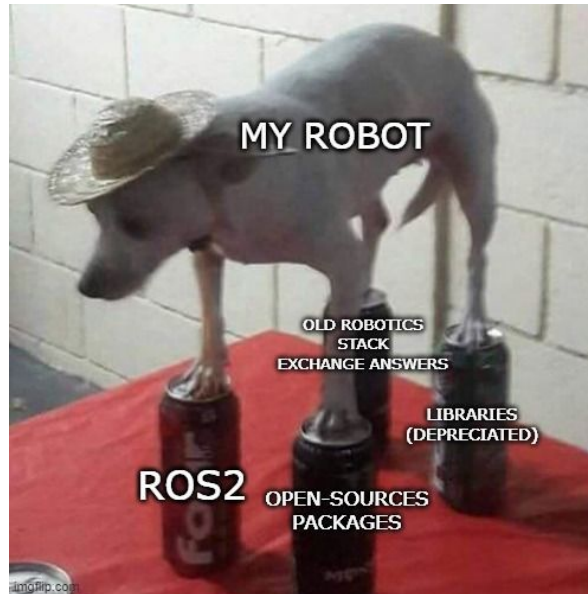
About bags: <https://docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Recording-And-Playing-Back-Data/Recording-And-Playing-Back-Data.html>

Let's work!

Let's create our own workspace!

<https://docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Creating-Your-First-ROS2-Package.html>

<https://docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Py-Publisher-And-Subscriber.html>



Thank You...

...for your attention

